MINI DINO GAME

A single-button endless runner on an Arduino Uno and a 16x2 LCD



Playing - dino (left) + cactus scrolling, score top-right

The game running on the 16x2 character LCD

Project	Mini Dino Game - Arduino Uno based endless runner
Hardware	Arduino Uno R3, 16x2 I2C LCD, KY-040 rotary encoder, piezo buzzer
Build type	100% solderless (breadboard and jumper wires only)
Cost	Rs. 775 for the complete bill of materials
Software	Arduino IDE, LiquidCrystal_I2C library, ~170 lines of C++
Prepared for	Team reference and viva preparation

Contents

Contents 2

1. Abstract 3

2. Introduction 3

3. Objectives and scope 3

4. System overview 3

5. Hardware components 4

6. Circuit design and wiring 6

7. What each connection actually does 7

8. Software environment 8

9. Program structure 9

10. Code walkthrough 9

11. Game logic 10

12. Custom characters and CGRAM 13

13. The three screens 13

14. Design decisions and alternatives 14

15. Testing and validation 15

16. Troubleshooting 15

17. Limitations and future scope 15

18. Project build plan 16

19. Viva questions and answers 16

20. Conclusion 17

21. References 18

Appendix A. Full source code 19

Appendix B. Bill of materials 21

Appendix C. Glossary 21

1. Abstract

This document describes the design, construction and working of a Mini Dino Game built on an Arduino Uno. The project reproduces the well-known Chrome offline dinosaur game on a 16x2 character LCD. Because a character LCD cannot display large graphics, the dinosaur and the cactus are drawn as custom 5 by 8 pixel characters stored in the LCD controller's own character memory (CGRAM). A single rotary-encoder push button starts the game, makes the dinosaur jump and restarts the game after a collision. A piezo buzzer gives audible feedback on a jump and on a crash.

The whole build is solderless: every module plugs into a breadboard or directly into the Uno's headers, and jumper wires make the connections. The complete bill of materials is Rs. 775. The software is a single Arduino sketch of about 170 lines that uses a three-state machine, a hardware interrupt for the button and a non-blocking redraw scheme that keeps the character LCD from flickering.

2. Introduction

The Chrome dinosaur is the small game that appears in Google Chrome when there is no internet connection. The player presses the space bar to make a dinosaur jump over cacti; the score rises the longer the player survives. It is a classic endless runner: there is no finish line, only a score and an eventual collision.

Reproducing that game on an Arduino is a useful exercise because it forces several embedded concepts to be used together in one small system:

- Inter-Integrated Circuit (I2C) communication, to drive a display with only two signal wires.
- External hardware interrupts, to react to a button press instantly instead of polling it.
- Character generation (CGRAM), to create graphics on a text-only display.
- Tone generation on a digital pin, to produce sound from a buzzer.
- A finite state machine, to organise the game into welcome, playing and game-over phases.

The result is small enough to build in an afternoon and cheap enough to be a student project, yet it touches almost every part of the Arduino platform.

3. Objectives and scope

The project had four concrete objectives:

1. **Reproduce the endless-runner mechanic** on a 16x2 character LCD, with a dinosaur that jumps over cacti and a score that increases.
2. **Use only solderless, low-cost modules** so the project can be assembled and dismantled without a soldering iron.
3. **Demonstrate four embedded techniques** in one build: I2C, external interrupts, CGRAM custom characters and tone generation.
4. **Keep the bill of materials under Rs. 800** so the project stays within a student budget.

Scope. The game is deliberately small. It has one obstacle type (a cactus), one control (a single push button) and one score. There are no levels, no sound effects beyond two beeps, and no persistent high score.

Out of scope. Colour graphics, multi-button control, sound melodies, EEPROM high-score storage and battery operation are all possible extensions but were not part of this build. They are discussed in section 17.

4. System overview

The system has one controller and three peripherals. The Arduino Uno runs the game program. The LCD is the display, the rotary encoder is the input and the buzzer is the output. A 5 V USB supply powers everything; the Uno's onboard regulator and the USB input provide the 5 V rail that the peripherals share.

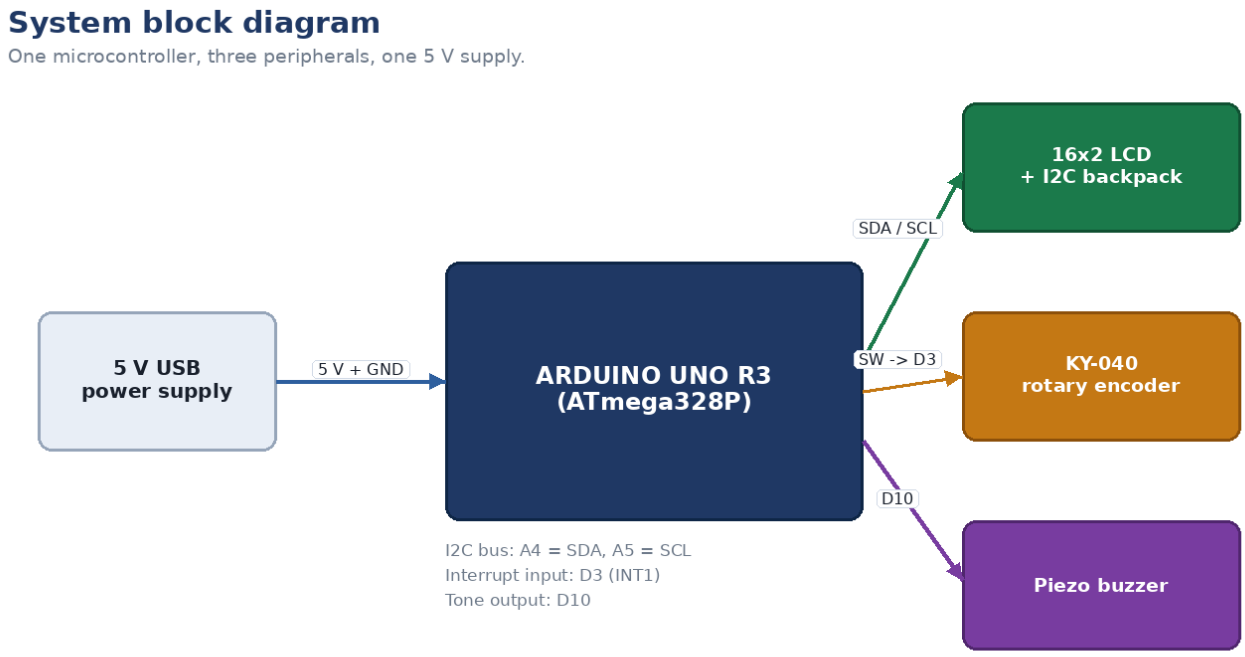


Figure 1 - System block diagram. The Uno is the only intelligent part; the peripherals are driven by it.

Only seven signal and power wires leave the Uno: four to the LCD (VCC, GND, SDA, SCL), two to the encoder (SW, GND, plus its 5 V feed) and two to the buzzer (signal, GND). Ground is common to all four devices.

5. Hardware components

Each component is described below with the internals that matter for the project, and the reason it was chosen.

5.1 Arduino Uno R3

The Uno is a microcontroller board built around the ATmega328P. It provides 14 digital input/output pins, six analog inputs, a USB interface for programming and power, and a 5 V regulated rail that the peripherals use.

Parameter	Value	Why it matters here
Microcontroller	ATmega328P, 8-bit AVR	Runs the whole game
Clock	16 MHz	Enough for a smooth 230 ms game tick
Flash	32 KB	Sketch uses a small fraction of it
SRAM	2 KB	Holds the three 8-byte character arrays
Digital I/O	14 pins	D3 for the button, D10 for the buzzer
Analog input	6 pins, A0-A5	A4 and A5 double as the I2C bus
Operating voltage	5 V	Matches the LCD and encoder
External interrupts	D2 (INT0) and D3 (INT1)	The button uses D3

Table 1 - Arduino Uno R3 specifications relevant to this project.

5.2 16x2 LCD with I2C backpack

The display is a 16-column, 2-row character LCD driven by an HD44780-compatible controller. On its own that controller needs six to ten signal wires. A small adapter board soldered to the back of the LCD carries a PCF8574 I2C expander, which converts those parallel lines into a two-wire I2C bus. That reduces the wiring from ten wires to four: VCC, GND, SDA and SCL.

The expander has a 7-bit I2C address of 0x27 on most boards, or 0x3F on some. The address is set by three solder bridges (A0, A1, A2) on the adapter. If the screen stays blank after correct wiring, the address is the first thing to change in the sketch.

Parameter	Value
Display	16 characters x 2 rows
Character cell	5 x 8 pixels
Controller	HD44780 compatible, behind a PCF8574 I2C expander
Interface	I2C, 4 pins: GND, VCC, SDA, SCL
I2C address	0x27 (or 0x3F)
Supply	5 V
Custom characters	8 slots in CGRAM, 5 x 8 pixels each

Table 2 - LCD and I2C backpack specifications.

5.3 KY-040 rotary encoder

The KY-040 is an incremental rotary encoder with an integrated push switch. Rotation produces two quadrature outputs (CLK and DT); pressing the shaft closes a separate switch (SW). This project uses only the switch. The module carries 10 k pull-up resistors on its outputs, so the switch line is held high and is pulled low when the shaft is pressed.

Only five pins exist: CLK, DT, SW, + and GND. The + and GND pins must be connected because the onboard pull-ups draw their supply from them. The CLK and DT pins are left unconnected in this build.

Pin	Function	Connected in this project?
CLK	Quadrature output A	No - rotation is not used
DT	Quadrature output B	No - rotation is not used
SW	Push switch, active low	Yes - to D3
+	Supply for the pull-up resistors	Yes - to 5 V
GND	Ground	Yes - to GND

Table 3 - KY-040 pin usage. Only three of the five pins are used.

5.4 Piezo buzzer

The buzzer converts an electrical signal into sound. Two kinds exist and the difference matters: an **active** buzzer contains its own oscillator and beeps whenever it is powered, while a **passive** buzzer needs an external square wave and plays the frequency it is given. The active type is simpler, but the sketch cannot control its pitch. The passive type lets the sketch play a high note on a jump and a low note on a crash.

The sketch is written for a passive buzzer using the `tone()` function. With an active buzzer the game still works and still beeps, but both events sound the same.

5.5 Breadboard, jumper wires and power

A 400-point solderless breadboard provides the common ground rail and a place to mount the buzzer. Jumper wires come in two kinds, and the difference matters in practice: male-to-male wires link the Uno to the breadboard, while male-to-female wires are needed to reach a module whose pins are male, such as the LCD. The board is powered from a 5 V USB source - a computer during development, or a 5 V 2 A phone charger once the sketch is uploaded.

5.6 Electrical budget

The Uno's USB input can supply 500 mA, and a phone charger supplies more. The board and all three peripherals together draw a small fraction of that, which is why a single USB port powers the whole project comfortably.

Component	Typical current	Peak
Arduino Uno board and MCU	50 mA	80 mA
LCD controller (HD44780)	2 mA	3 mA
LCD backlight LEDs	25 mA	40 mA
PCF8574 I2C expander	0.1 mA	0.2 mA
KY-040 encoder	under 1 mA	1 mA
Piezo buzzer	25 mA	35 mA
Total (approximate)	**about 105 mA**	**about 160 mA**

Table 5 - Current consumption. Well inside the 500 mA a USB port provides.

Two conclusions follow. First, no external power supply or driver transistor is needed - every peripheral can be driven directly from a digital pin. Second, a 5 V 2 A phone charger is more than adequate once the sketch has been uploaded, which is how the project runs without a computer.

5.7 Pin allocation

Only four of the Uno's twenty I/O pins are used, and two of those are the hardware I2C pins. The rest stay free, which is what makes the extension ideas in section 17 realistic.

Pin	Direction	Connected to	Function
A4	bidirectional	LCD SDA	I2C data
A5	output	LCD SCL	I2C clock
D3	input, pull-up	Encoder SW	Button, INT1
D10	output	Buzzer +	Tone output
5V	power out	LCD VCC, encoder +	5 V rail
GND	ground	All four devices	Common return
D0, D1	reserved	USB-serial chip	Do not use - upload port
D2, D4-D9, D11-D13, A0-A3	free	nothing	Available for extension

Table 6 - Pin allocation. D0 and D1 must be left alone because they carry the USB upload link.

6. Circuit design and wiring

There are eight connections in total. The table below is the authoritative list; the diagram that follows shows the same information laid out physically.

From	To	Wire	Purpose
LCD VCC	Uno 5V	red	Powers the LCD logic and backlight
LCD GND	Uno GND	black	Return path for the LCD
LCD SDA	Uno A4	blue	I2C data line
LCD SCL	Uno A5	green	I2C clock line
Encoder SW	Uno D3	orange	Button signal (interrupt)
Encoder +	Uno 5V	red	Supply for the pull-ups

From	To	Wire	Purpose
Encoder GND	Uno GND	black	Ground reference
Buzzer +	Uno D10	purple	Tone signal
Buzzer -	Uno GND	black	Return path for the buzzer

Table 4 - The complete connection list.

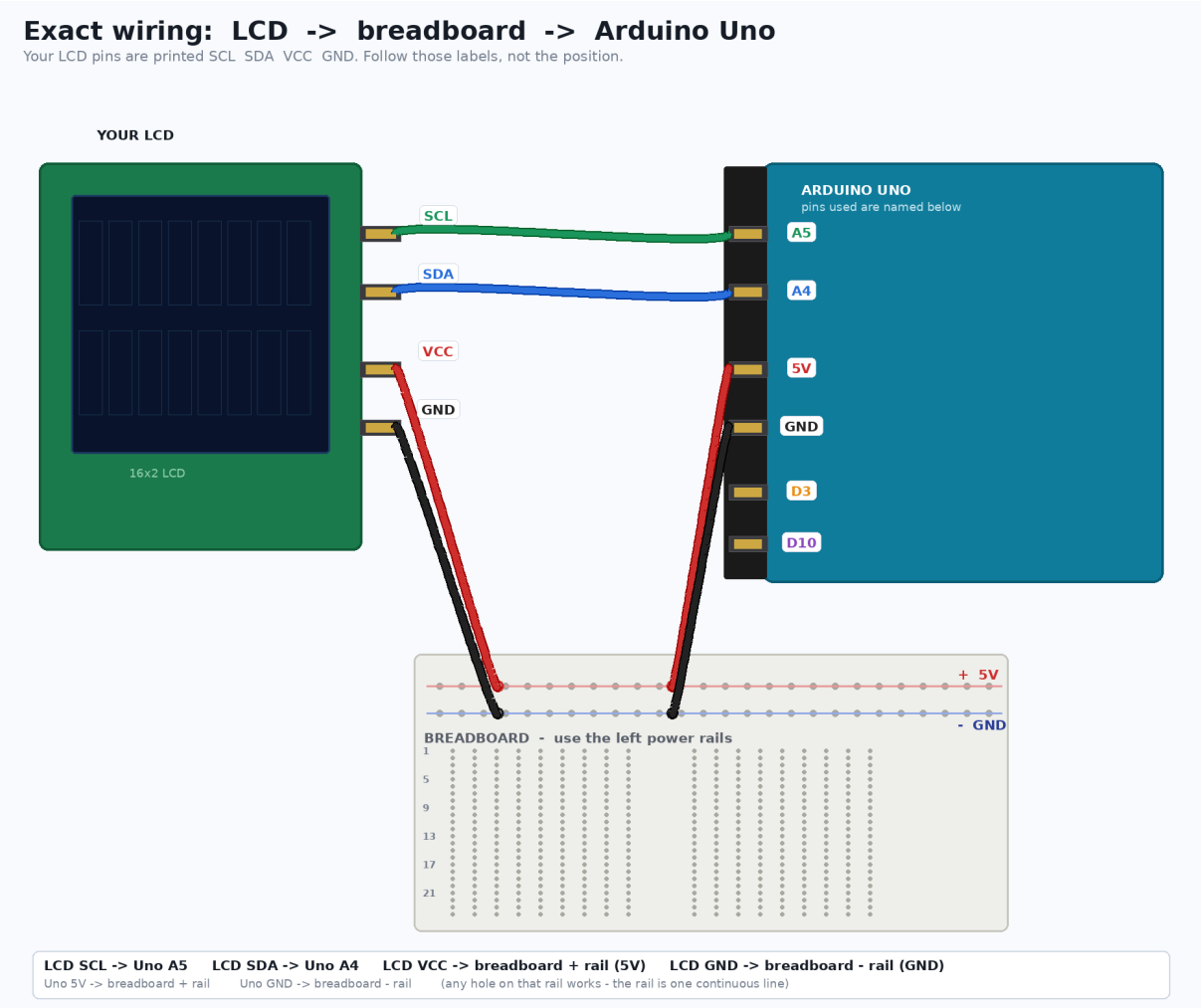


Figure 2 - Physical layout. The breadboard supplies the shared 5 V and ground rails.

Pin-order warning. The four pins on an I2C backpack are not in a fixed order across manufacturers. The board used here is printed SCL, SDA, VCC, GND on one face, which is the reverse of the more common GND, VCC, SDA, SCL. Always follow the word printed next to the pin, never the position.

7. What each connection actually does

Each wire has a specific electrical job. Understanding these is the core of a viva on this project.

LCD VCC to 5 V. The LCD controller and the backlight LEDs both run from 5 V. The backlight alone draws roughly 20 to 30 mA. Feeding 3.3 V instead is a common mistake: the logic may work but the backlight stays dim or off, and the display looks dead.

LCD GND to GND. Every signal is measured against this reference. Without a common ground the I2C lines have no reference and the display shows random blocks.

LCD SDA to A4. Serial Data. On the ATmega328P this is the I2C data line; it carries both the address of the device and the data bytes, in both directions.

LCD SCL to A5. Serial Clock. The master (the Uno) generates this clock and every device on the bus samples the data line on its edge. A4 and A5 are the hardware I2C pins, so the Wire library can use them without any software overhead.

Encoder SW to D3. The switch output. It rests high because of the module's pull-up and is pulled to ground when the shaft is pressed. D3 is one of only two pins on the Uno with an external interrupt (INT1), which is why the button is wired there rather than to an arbitrary pin.

Encoder + and GND. These power the 10 k pull-up resistors on the module. Without them the switch line floats and the button becomes unreliable, even though the Arduino's internal pull-up would otherwise help.

Buzzer + to D10. The tone signal. The sketch switches this pin on and off hundreds of times a second; the buzzer turns that into sound.

Buzzer - to GND. The return path. The polarity matters: a buzzer connected the wrong way round will not sound.

8. Software environment

The sketch is written for the Arduino IDE. Two libraries are required. **Wire** is bundled with the IDE and implements the I2C bus. **LiquidCrystal_I2C** sits on top of Wire and provides the familiar `print()`, `setCursor()` and `createChar()` calls for an I2C-backed HD44780 display.

The upload sequence is:

1. Install the Arduino IDE and open it.
2. Select Tools > Board > Arduino Uno and Tools > Port > the port that appeared when the board was plugged in.
3. Install LiquidCrystal I2C from Tools > Manage Libraries.
4. Open MiniDino.ino. The sketch folder must be named Dino.
5. Click Verify, wait for Done compiling, then click Upload.

8.1 How the I2C bus works

I2C (Inter-Integrated Circuit) is a two-wire serial bus. One device - the master - drives the clock line (SCL) and both devices share the data line (SDA). Every transaction begins with a start condition, then a 7-bit address plus a read/write bit, then the data bytes, and ends with a stop condition. The addressed device acknowledges each byte by pulling the data line low for one clock pulse.

Both lines are open-drain and are held high by pull-up resistors. On an LCD backpack those resistors are already fitted, which is why the module needs no extra components. Because the bus is shared, several devices can sit on the same two wires as long as each has a different address - the LCD's PCF8574 is at 0x27 or 0x3F, leaving plenty of addresses free for sensors later.

Term	Meaning
Master	The device that generates the clock - here, the Uno
Slave	The addressed device - here, the PCF8574 on the LCD backpack
SDA	Serial data line, bidirectional
SCL	Serial clock line, driven by the master
Address	7-bit identity of a slave; 0x27 or 0x3F for this backpack
ACK	The slave pulling SDA low to confirm it received a byte
Speed	100 kHz standard mode, 400 kHz fast mode

Table 7 - I2C terms used in this project.

The practical benefit is pin economy. A parallel HD44780 interface needs six data and control lines plus power; the same display over I2C needs two. Those saved pins are what make it possible to add an encoder and a buzzer to the same board without running out.

9. Program structure

The sketch is one file organised into four layers: configuration, state, helpers and the two mandatory Arduino functions.

Layer	Contents	Purpose
Configuration	Pin defines, character arrays, timing constants	One place to change pins and speed
State	state, cactusCol, dinoRow, jumping, score, frame	Everything the game remembers between loop passes
Helpers	showStart, startGame, gameOver, jump, handleButton, renderPlay	One job each, called from loop
Arduino core	setup() and loop()	Initialisation once; the game tick forever

Table 8 - Structure of MiniDino.ino.

10. Code walkthrough

10.1 setup()

setup() runs once at power-on. It configures the button pin with an internal pull-up, configures the buzzer pin as an output, attaches the interrupt, initialises the LCD and uploads the three custom characters into CGRAM.

```
pinMode(SW, INPUT_PULLUP);
pinMode(BUZZER, OUTPUT);
attachInterrupt(digitalPinToInterrupt(SW), onButton, FALLING);
lcd.init(); lcd.backlight();
lcd.createChar(0, dinoRunA); lcd.createChar(1, dinoRunB); lcd.createChar(2, cactusCh);
```

The interrupt is set to fire on the FALLING edge, that is, at the moment the pin goes from high to low. That is exactly the instant the button is pressed, so the game reacts immediately rather than waiting for the next pass of loop().

10.2 The interrupt handler

The handler does almost nothing, which is deliberate. An interrupt service routine should be short: it sets a flag and returns, and loop() does the real work.

```
volatile bool buttonPressed = false;
void onButton() { buttonPressed = true; }
```

The variable is declared `volatile` because it is modified inside an interrupt and read outside one. Without that keyword the compiler may cache the value in a register and loop() would never see the change.

10.3 loop()

loop() repeats forever. Each pass does three things: act on a pending button press, advance the game if it is running, and redraw what changed.

```
if (buttonPressed) { buttonPressed = false; handleButton(); }
if (state == ST_PLAY) {
  if (jumping && millis() - jumpStart >= JUMP_MS) jumping = false;
  dinoRow = jumping ? 0 : 1;
  if (millis() - lastMove >= MOVE_MS) { lastMove = millis(); cactusCol--; ... }
  if (state == ST_PLAY) renderPlay();
}
delay(10);
```

The timing uses `millis()` rather than `delay()`. This is the single most important design decision in the sketch: because the game never blocks, the button stays responsive even while the cactus is moving.

10.4 handleButton()

One physical button has three different meanings depending on the state. The function is therefore a small dispatch:

```
if (state == ST_START) startGame();
else if (state == ST_PLAY) jump();
else showStart();
```

10.5 renderPlay()

A character LCD is slow. Clearing the whole screen and redrawing it every frame produces visible flicker. renderPlay() therefore compares each element with its previous value and redraws only what changed - the old cactus cell, the new cactus cell, the dinosaur and the score. Four cached variables hold the previous values.

10.6 gameOver()

On a collision the state changes to ST_OVER, a low tone is played, the screen is cleared and the final score is printed. The game does not reset itself; it waits for a press, which is what makes the restart deliberate.

10.7 Interrupts in detail

An interrupt is a hardware signal that makes the processor stop what it is doing, run a short function called an interrupt service routine, and then resume exactly where it left off. Without an interrupt, a button press would only be noticed when loop() next reached the line that reads the pin - which could be several milliseconds later, or not at all if the code were busy.

The ATmega328P has two external interrupt lines: INT0 on D2 and INT1 on D3. The sketch uses D3. The trigger is set to FALLING, meaning the routine runs at the moment the pin drops from high to low - which is precisely when the button closes.

```
attachInterrupt(digitalPinToInterrupt(SW), onButton, FALLING);
```

Three rules make an interrupt service routine safe, and this sketch follows all three:

- **Keep it short.** The routine only sets a flag. All real work happens in loop().
- **Never call delay().** A delay inside an interrupt stalls the entire system.
- **Mark shared variables volatile.** The flag is written in the interrupt and read in loop(), so the compiler must not cache it.

There is a second, quieter benefit: because the button is handled by an interrupt, the game loop is free to spend its time on the display. The redraw is slow on a character LCD, and without the interrupt a press during a redraw would be lost.

11. Game logic

11.1 The state machine

The game has three states and one button, so every press must be interpreted in context. The state machine below is the complete specification of that behaviour.

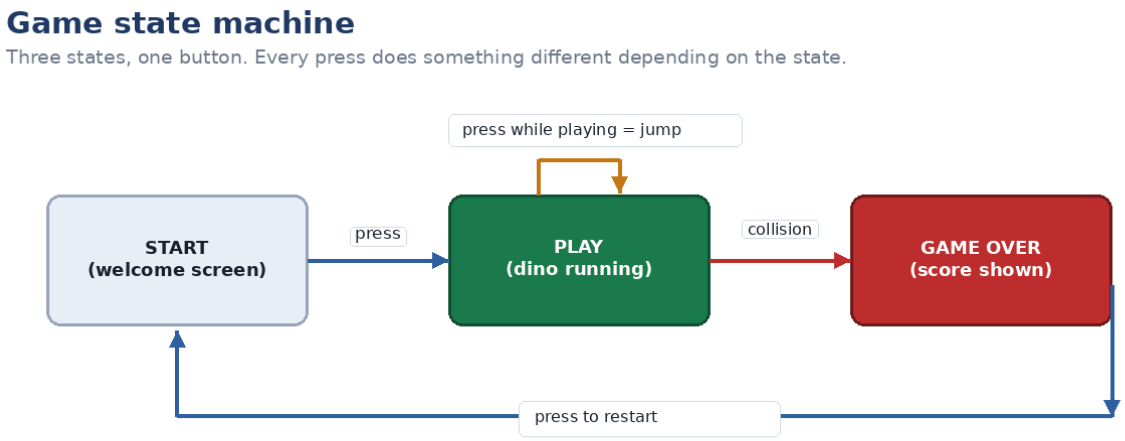


Figure 3 - State machine. A press means start, jump or restart depending on the current state.

State	What the screen shows	What a press does
START	Welcome text	Begins the game
PLAY	Dino, cactus, score	Makes the dino jump
OVER	GAME OVER and final score	Returns to START

Table 9 - Behaviour of the single button in each state.

11.2 Movement model

The play field is the bottom row of the LCD, which is 16 character cells wide. The dinosaur occupies column 0 and never moves horizontally. The cactus has a column number that decreases by one every 230 milliseconds. When it falls below zero it has passed the dinosaur, so it is respawned at column 15 and the score increases by one.

Jumping is modelled as a row change rather than a pixel animation, because the display has only two rows. On a press the dinosaur is drawn on row 0 for 450 milliseconds and then returns to row 1. Two constants control the feel of the game: MOVE_MS sets the speed and JUMP_MS sets how long the dinosaur stays in the air.

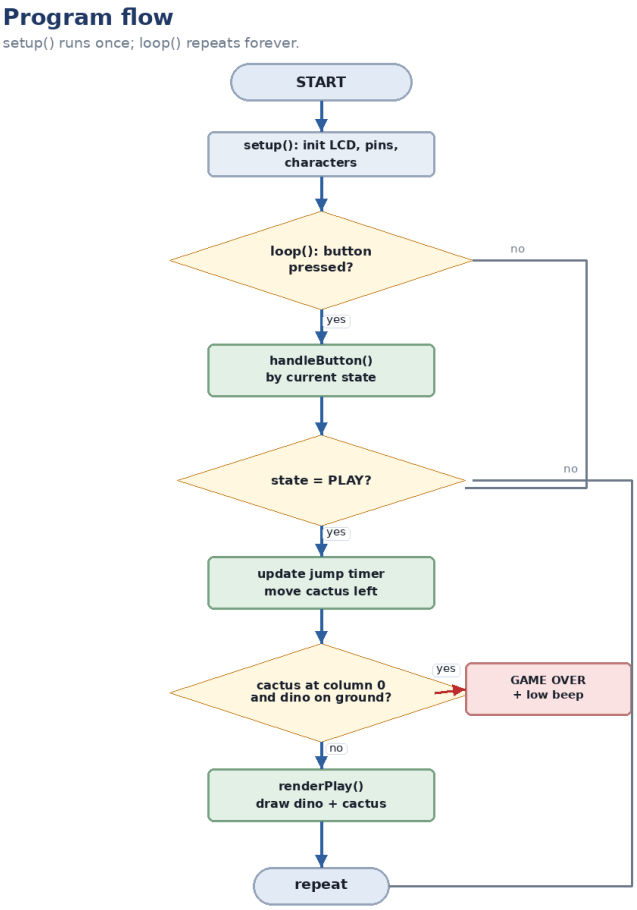


Figure 4 - Program flow through setup() and one pass of loop().

11.3 Collision detection

Collision is a single comparison. After the cactus moves, the sketch checks whether the cactus has reached column 0 while the dinosaur is on the ground row. If both are true, the cactus has hit the dinosaur and the game ends. If the dinosaur is in the air at that moment, the cactus passes underneath safely.

```
if (cactusCol < 0) { cactusCol = 15; score++; }
else if (cactusCol == 0 && dinoRow == 1) { gameOver(); }
```

This is a coarse test - it treats the dinosaur as a single character cell rather than a shape - but on a 16-column display that is exactly the right level of precision, and it is what makes the game feel fair.

11.4 Scoring

The score is the number of cacti successfully passed. It is displayed in the top-right corner of the top row, prefixed with an S. It is redrawn only when its value changes, so the digits do not flicker while the cactus is moving.

11.5 Theory of operation - one complete game cycle

It helps to read the whole system as a single sequence. When power is applied, setup() runs once: the button pin is configured, the buzzer pin becomes an output, the interrupt is attached, the LCD is initialised and the three custom characters are written into CGRAM. The start screen is then printed and loop() begins.

In the START state nothing moves. The cactus sits at column 15, off the visible game logic, and the screen shows the welcome text. The only thing that can change is the button flag, set by the interrupt.

When the button is pressed, handleButton() sees the START state and calls startGame(). The score resets, the cactus is placed at column 15 and the state becomes PLAY. On the very next pass of loop(), the play branch begins executing: every 230 milliseconds the cactus column decreases by one, and after each step

renderPlay() redraws only the two cells that changed.

The player now presses the button to jump. The interrupt sets the flag, handleButton() calls jump(), and the jump routine starts a 450 millisecond timer and plays a 700 Hz tone. For the duration of that timer the dinosaur is drawn on the top row instead of the ground row. If the cactus reaches column 0 during that window, the collision test fails to fire and the cactus passes safely, which is the entire skill of the game.

If instead the cactus reaches column 0 while the dinosaur is on the ground, gameOver() runs: a 125 Hz tone sounds, the screen is cleared and the final score is printed. The state becomes OVER and the game stops advancing. The next press returns to the start screen and the cycle can begin again.

12. Custom characters and CGRAM

An HD44780 character LCD has a small amount of RAM reserved for user-defined characters, called CGRAM (character generator RAM). It holds eight characters, each five pixels wide and eight pixels tall. Once a bitmap is written into a slot, that slot can be printed like any ordinary character.

The sketch defines three characters: two dinosaur run frames and one cactus. The two dinosaur frames differ only in their bottom two rows, which is what produces the running illusion when they alternate every other frame.

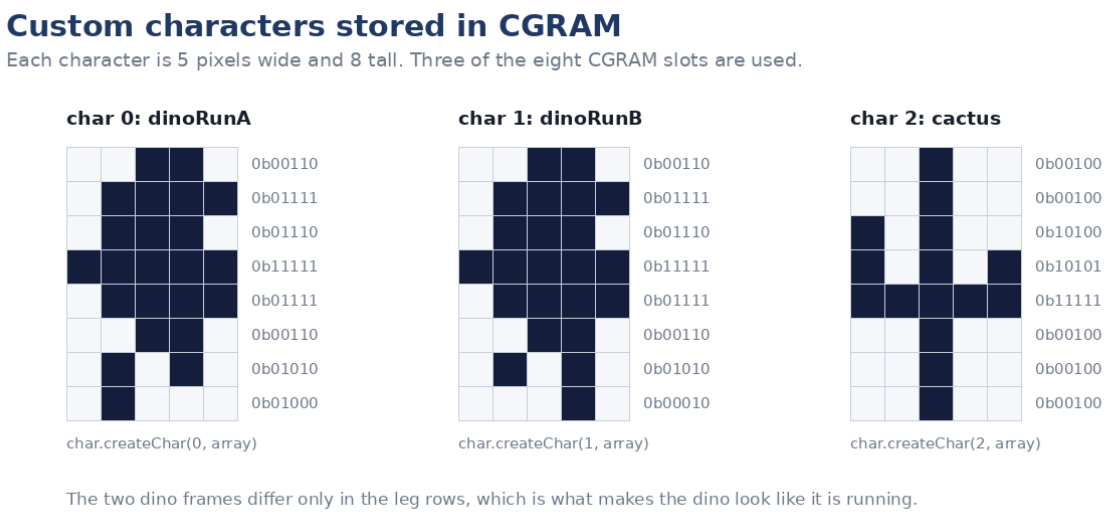


Figure 5 - The three custom characters and their byte values.

Each row of a character is one byte, and only the lower five bits are used. Reading a row as five bits from left to right gives the pixel pattern directly, which is why the arrays in the sketch look like small pictures.

```
lcd.createChar(0, dinoRunA); // slot 0
lcd.write(byte(0)); // print it
```

13. The three screens

The display has two rows of sixteen characters. The bottom row is the play field; the top row carries the score and the messages. Nothing else is ever drawn.



Start screen

Start screen - the game waits for a press.



Playing - dino (left) + cactus scrolling, score top-right

Playing - dino at the left, cactus moving in from the right, score top-right.



Game over

Game over - the final score and a low beep.

14. Design decisions and alternatives

Every part of this build could have been done differently. The table records what was chosen, what was rejected and why - which is exactly the sort of reasoning a viva rewards.

Decision	Chosen	Rejected alternative	Reason
Display interface	I2C backpack	Parallel HD44780	2 wires instead of 6-10; frees pins
Display type	16x2 character LCD	128x64 OLED	Cheaper (Rs. 185 vs Rs. 260+) and matches the budget
Graphics method	CGRAM custom characters	Full bitmap graphics	A character LCD physically cannot draw sprites
Button handling	Hardware interrupt on D3	Polling in loop()	Instant response; no press is missed
Timing	millis() comparison	delay()	delay() blocks the button and freezes the game
Redraw strategy	Cached cells, redraw changed only	Clear and redraw every frame	Full clear causes visible flicker
Input device	Single encoder push button	Separate push buttons	One control is enough; fewer parts and wires
Buzzer type	Passive piezo (for tone())	Active buzzer	Passive allows two different pitches

Decision	Chosen	Rejected alternative	Reason
Assembly	Solderless breadboard	Soldered perfboard	Reusable, no tools, safe for students
Logic level	5 V throughout	3.3 V	The LCD and encoder are specified for 5 V

Table 10 - Design decisions, alternatives considered and the reason for each choice.

15. Testing and validation

The build was tested in stages, so that a fault could be traced to one subsystem at a time.

Test	Method	Expected result
Power	Connect USB, observe the board LEDs	Power LED lit; L LED blinking (factory sketch)
Backlight	Power the LCD only	Screen glows uniformly
I2C bus	Run an I2C scanner sketch	Device found at 0x27 or 0x3F
Contrast	Turn the pot on the backpack	Text becomes readable
Button	Press the encoder and watch the Serial Monitor	One clean event per press
Buzzer	Briefly touch + to 5 V and - to GND	Continuous tone (active type)
Full game	Upload and play	Start, jump, score, collision, restart all work

Table 11 - Staged test plan. Testing power and backlight before the bus saves a lot of guesswork.

16. Troubleshooting

The faults below account for almost every failure seen while building this project.

Symptom	Cause	Fix
Screen completely dark	No power to the LCD, or VCC/GND swapped	Check VCC on 5 V and GND on GND; verify the backlight glows
Backlight on, no text	Contrast wrong, or no sketch uploaded	Turn the blue pot; upload the sketch
Only blocks on screen	Contrast far off, or wrong I2C address	Adjust the pot; change 0x27 to 0x3F
Button does nothing	SW not on D3, or encoder not powered	Move SW to D3; connect encoder + and GND
Buzzer silent	Polarity reversed, or active buzzer	Swap the two leads; expect one tone only if active
Upload fails	Wrong port, or a charge-only cable	Select the correct port; use a data cable
Screen flickers	Full clear on every frame	Use the cached redraw scheme in renderPlay()

Table 12 - Troubleshooting reference.

17. Limitations and future scope

The build works, but it is deliberately minimal. Being able to name its limits, and the fix for each, is worth marks in a viva.

Limitation	Reason	Possible extension
Graphics are only 5x8 characters	The LCD is a character display	Swap for a 128x64 OLED and draw full sprites
One obstacle type	Only three CGRAM slots are used	Use the remaining five slots for birds and rocks
No high score	Nothing is stored after power-off	Save the best score in EEPROM

Limitation	Reason	Possible extension
Single button	One control only	Use the encoder's CLK/DT for a second action
Two tones only	The sketch plays two fixed notes	Use a passive buzzer and play a melody
Tethered to USB	Power comes from USB	Run from a 9 V battery through the barrel jack

Table 13 - Limitations and the natural next step for each.

18. Project build plan

The build is short enough to complete in one sitting, but it should be done in this order. Testing power and the backlight before connecting anything else saves the most time, because a dark screen has only two possible causes at that stage.

Phase	Tasks	Time
1. Procurement	Buy all seven items from Bhagirath Palace, Chandni Chowk	1-2 hours
2. Power check	Plug the Uno into USB; confirm the power LED and the L LED	5 min
3. LCD wiring	Connect VCC, GND, SDA, SCL; confirm the backlight glows	15 min
4. Encoder wiring	Connect SW to D3, + to 5 V, GND to GND	10 min
5. Buzzer wiring	Connect + to D10, - to GND; test by touching 5 V briefly	5 min
6. Toolchain	Install Arduino IDE and the LiquidCrystal I2C library	20 min
7. Upload	Open MiniDino.ino, select board and port, verify, upload	10 min
8. Commissioning	Adjust contrast, play, tune MOVE_MS and JUMP_MS	15 min
Total		**about 3 hours**

Table 14 - Build sequence and realistic timings.

Order matters. If the LCD is wired last, a blank screen could be caused by the bus, the address, the contrast or the code. Wiring it first means the backlight test isolates power immediately, and every later fault is then known to be a signal problem.

19. Viva questions and answers

These are the questions most likely to be asked about this build, with the answers in the order they should be given.

Q1. Why use an I2C LCD instead of a normal 16x2 LCD?

A normal HD44780 LCD needs six to ten signal wires. The I2C backpack puts a PCF8574 expander on the back, which converts that parallel interface into a two-wire bus. So the display costs only four wires in total - VCC, GND, SDA and SCL - and leaves the other pins free.

Q2. What is the I2C address of your display and how did you find it?

0x27 on most backpacks, 0x3F on some. It is set by three solder bridges labelled A0, A1 and A2 on the adapter. If the display is blank with correct wiring, an I2C scanner sketch reports the address actually in use.

Q3. Why is the button connected to D3 and not to any other pin?

Because D3 is one of only two pins on the ATmega328P with an external interrupt line (INT1; D2 is INT0). Using an interrupt means the press is caught the instant it happens, instead of being missed between passes of loop().

Q4. What is debouncing and does your code do it?

A mechanical switch bounces for a few milliseconds when pressed, producing several edges. Here the risk is several interrupts per press. The code guards against this by acting only on the first edge while the state is changing, and by making the resulting action idempotent - a second jump request while already jumping is ignored.

Q5. What is the difference between an active and a passive buzzer?
An active buzzer has a built-in oscillator and beeps whenever it is powered, at a fixed pitch. A passive buzzer needs an external square wave, so the microcontroller controls the frequency. This sketch uses tone(), which suits a passive buzzer and gives two different pitches.

Q6. What is CGRAM and why did you need it?
CGRAM is character generator RAM inside the LCD controller. It holds eight user-defined characters of 5x8 pixels. A character LCD cannot draw arbitrary graphics, so the dinosaur and cactus had to be defined as custom characters and printed like normal letters.

Q7. How is the running animation produced?
Two nearly identical dinosaur characters are stored in CGRAM. They differ only in the bottom two rows, which represent the legs. The sketch alternates between them every other frame, which the eye reads as running.

Q8. How does collision detection work?
The cactus has a column number and the dinosaur is fixed at column 0. After each move the code checks whether the cactus has reached column 0 while the dinosaur is on the ground row. If both are true, the game ends.

Q9. How is the score calculated?
It is the number of cacti successfully passed. Each time the cactus leaves the screen it is respawned at the right edge and the score increases by one.

Q10. Why does the sketch use millis() instead of delay()?
delay() blocks the processor, so the button would be ignored for the whole delay. millis() lets loop() run continuously and simply checks whether enough time has passed. This keeps the game responsive and the animation smooth.

Q11. Why is the button variable declared volatile?
Because it is modified inside an interrupt service routine and read outside it. Without volatile, the compiler may keep the value in a register and loop() would never see the update.

Q12. Why does the display not flicker?
Because the sketch never clears the whole screen. It compares each element with its previous value and rewrites only the cells that changed. A full clear on every frame is what causes the familiar LCD flicker.

Q13. What would you change to make the game harder?
Reduce MOVE_MS, which shortens the interval between cactus steps. The value is a single constant at the top of the sketch, so the difficulty can be changed without touching the game logic.

Q14. What happens if you connect the LCD to 3.3 V?
The logic may partly work but the backlight will be dim or off and contrast will be wrong, so the display appears dead. The LCD and its backpack are specified for 5 V, which is why VCC goes to the 5 V pin and never to 3.3 V.

Q15. How would you add a second obstacle type?
There are five unused CGRAM slots. Define a new character in one of them, add it to the random selection when the cactus respawns, and give it its own collision width. The rest of the logic is unchanged.

Q16. What is the total cost of the build?
Rs. 775 for the complete bill of materials: Arduino Uno Rs. 280, LCD Rs. 185, encoder Rs. 150, breadboard Rs. 50, jumper wires Rs. 100 and buzzer Rs. 10.

20. Conclusion

The Mini Dino Game demonstrates that a complete, playable game can be built from four low-cost modules and about 170 lines of code. More importantly, it shows how several independent embedded techniques combine into one working system: I2C to talk to a display over two wires, an external interrupt to catch a button press instantly, CGRAM to create graphics on a text-only screen, and non-blocking timing to keep everything responsive at once.

The build is solderless, costs Rs. 775 and can be assembled in under an hour. Its limitations are real - the graphics are eight pixels tall and there is only one obstacle - but each limitation has a clear, small next step, which is what makes the project a good starting point rather than a dead end.

21. References

- Arduino Uno R3 product reference and pinout, arduino.cc.
- ATmega328P datasheet, Microchip Technology - external interrupt and I2C sections.
- HD44780U LCD controller datasheet - character generator RAM (CGRAM) and instruction set.
- PCF8574 remote 8-bit I/O expander datasheet, Texas Instruments.
- LiquidCrystal_I2C library documentation, Frank de Brabander.
- Arduino Wire library reference, arduino.cc.
- Source sketch and project files: github.com/krishsharma-code/Dino-game.

Appendix A. Full source code

The complete sketch, MiniDino.ino, is reproduced below. It is about 170 lines and compiles for an Arduino Uno with the LiquidCrystal I2C library installed. The comments are part of the original file.

```
/*
 *  MINI DINO  -  Arduino Uno + 16x2 I2C LCD
 *  ----------------------------------------------------------------
 *  A tiny Chrome-dino style game for a cheap 16x2 character LCD.
 *  The dino and the cactus are drawn with custom 5x8 characters
 *  (CGRAM). Press the KY-040 encoder button to jump over the cactus.
 *
 *  PARTS (all solderless, plug into breadboard / Uno sockets):
 *    Arduino Uno R3  |  16x2 I2C LCD (PCF8574)  |  KY-040 rotary encoder
 *    Passive piezo buzzer  |  solderless breadboard  |  jumper wires
 *
 *  WIRING:
 *    LCD    : VCC->5V, GND->GND, SDA->A4, SCL->A5
 *    KY-040 : SW->D3, VCC->5V, GND->GND     (DT->D4, CLK->D2 wired, unused)
 *    Buzzer : + -> D10, - -> GND
 *
 *  LIBRARY NEEDED (install from Arduino IDE > Library Manager):
 *    "LiquidCrystal I2C" by Frank de Brabander
 *
 *  NOTE: if the screen stays blank, change the address below
 *        from 0x27 to 0x3F (or run an I2C scanner).
 */

#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);   // try 0x3F if 0x27 shows nothing

// ---- pins ----
#define SW      3       // KY-040 push switch  -> jump / start
#define BUZZER  10      // passive piezo buzzer

// ---- custom characters (5 wide x 8 tall) ----
// 1 = lit pixel, 0 = blank
byte dinoRunA[8] = {0b00110, 0b01111, 0b01110, 0b11111, 0b01111, 0b00110, 0b01010, 0b01000};
byte dinoRunB[8] = {0b00110, 0b01111, 0b01110, 0b11111, 0b01111, 0b00110, 0b01010, 0b00010};
byte cactusCh[8] = {0b00100, 0b00100, 0b10100, 0b10101, 0b11111, 0b00100, 0b00100, 0b00100};

// ---- game state ----
#define ST_START 0
#define ST_PLAY  1
#define ST_OVER  2
byte state = ST_START;

int  cactusCol = 15;          // cactus column (right edge = 15)
int  dinoRow   = 1;           // 1 = on the ground, 0 = in the air
bool jumping   = false;
unsigned long jumpStart = 0;
const int JUMP_MS = 450;      // how long a jump lasts

int  score = 0;
int  frame = 0;
unsigned long lastMove = 0;
const int MOVE_MS = 230;      // ms per cactus step (game speed)

volatile bool buttonPressed = false;

// cached values so we redraw only what changed (no flicker)
int prevCactus   = -1;
int prevDinoRow  = -1;
int prevDinoChar = -1;
int prevScore    = -1;

void onButton() { buttonPressed = true; }

void showStart() {
  lcd.clear();
  lcd.setCursor(2, 0); lcd.print("MINI DINO");
  lcd.setCursor(1, 1); lcd.print("Press to start");
  state = ST_START;
}

void startGame() {
  cactusCol = 15; dinoRow = 1; jumping = false; score = 0; frame = 0;
  prevCactus = -1; prevDinoRow = -1; prevDinoChar = -1; prevScore = -1;
  lcd.clear();
  lastMove = millis();
  state = ST_PLAY;
}

void gameOver() {
  tone(BUZZER, 125, 400);
  lcd.clear();
  lcd.setCursor(3, 0); lcd.print("GAME OVER");
  lcd.setCursor(4, 1); lcd.print("Score:"); lcd.print(score);
  state = ST_OVER;
}
void jump() {
```

```
  if (!jumping) {
    jumping = true;
    jumpStart = millis();
    tone(BUZZER, 700, 80);
  }
}

void handleButton() {
  if (state == ST_START)      startGame();
  else if (state == ST_PLAY)  jump();
  else                        showStart();
}

void renderPlay() {
  byte d = ((frame / 2) % 2) ? 1 : 0;   // alternate the two run frames

  // --- cactus ---
  if (cactusCol != prevCactus) {
    if (prevCactus >= 0 && prevCactus <= 15) { lcd.setCursor(prevCactus, 1); lcd.print(' '); }
    lcd.setCursor(cactusCol, 1);
    lcd.write(byte(2));
    prevCactus = cactusCol;
  }

  // --- dino ---
  if (dinoRow != prevDinoRow || d != prevDinoChar) {
    if (prevDinoRow >= 0) { lcd.setCursor(0, prevDinoRow); lcd.print(' '); }
    lcd.setCursor(0, dinoRow);
    lcd.write(d);
    prevDinoRow = dinoRow; prevDinoChar = d;
  }

  // --- score, top-right, redrawn only when it changes ---
  if (score != prevScore) {
    lcd.setCursor(12, 0); lcd.print("    ");
    lcd.setCursor(12, 0); lcd.print("S"); lcd.print(score);
    prevScore = score;
  }

  frame++;
}

void setup() {
  pinMode(SW, INPUT_PULLUP);
  pinMode(BUZZER, OUTPUT);
  attachInterrupt(digitalPinToInterrupt(SW), onButton, FALLING);

  lcd.init();
  lcd.backlight();
  lcd.createChar(0, dinoRunA);
  lcd.createChar(1, dinoRunB);
  lcd.createChar(2, cactusCh);

  showStart();
}

void loop() {
  if (buttonPressed) { buttonPressed = false; handleButton(); }

  if (state == ST_PLAY) {
    // jump timing
    if (jumping && millis() - jumpStart >= JUMP_MS) jumping = false;
    dinoRow = jumping ? 0 : 1;

    // move the cactus one step left
    if (millis() - lastMove >= MOVE_MS) {
      lastMove = millis();
      cactusCol--;
      if (cactusCol < 0) {
        cactusCol = 15;      // passed safely -> new cactus, score up
        score++;
      } else if (cactusCol == 0 && dinoRow == 1) {
        gameOver();          // hit the cactus on the ground
      }
    }

    if (state == ST_PLAY) renderPlay();
  }

  delay(10);
}
```

Appendix B. Bill of materials

Prices are the street rates paid at Bhagirath Palace, Chandni Chowk, Delhi, for single quantities. The total of Rs. 775 sits inside the Rs. 600-800 budget set for the project.

Item	Qty	Price (Rs.)
Arduino Uno R3 with USB cable	1	280
16x2 LCD with I2C backpack	1	185
KY-040 rotary encoder module	1	150
Solderless breadboard, 400 point	1	50
Jumper wires, male-to-male	1	50
Jumper wires, male-to-female	1	50
Piezo buzzer	1	10
Total	**7**	**775**

Table 15 - Bill of materials.

Consumables and tools are not included: a computer with the Arduino IDE for the one-time upload, and a small flat screwdriver to turn the contrast potentiometer. Neither is part of the product.

Appendix C. Glossary

Term	Meaning
Arduino Uno	A microcontroller board based on the ATmega328P, programmed over USB
ATmega328P	The 8-bit AVR microcontroller that runs the sketch
Sketch	An Arduino program; the file with the .ino extension
I2C	A two-wire serial bus using SDA and SCL
SDA / SCL	I2C data line and clock line; A4 and A5 on the Uno
PCF8574	The I2C expander chip on the LCD backpack
HD44780	The controller family used by character LCDs
CGRAM	Character generator RAM; holds up to 8 user-defined 5x8 characters
Custom character	A 5x8 pixel glyph defined by the sketch and printed like a letter
Interrupt	A hardware signal that makes the CPU run a short routine immediately
ISR	Interrupt Service Routine; the function that runs on an interrupt
volatile	A keyword telling the compiler a variable may change outside normal flow
Pull-up resistor	Holds a signal line high until something pulls it low
Active buzzer	A buzzer with a built-in oscillator; fixed pitch
Passive buzzer	A buzzer that needs an external frequency; pitch is controllable
tone()	An Arduino function that generates a square wave of a given frequency
millis()	Returns milliseconds since power-on; used for non-blocking timing
State machine	A design where behaviour depends on a current mode or state
Solderless	Connected by plugging into a breadboard or headers; no soldering

Table 16 - Glossary of terms used in this document.